TESTNG

CLASS 2 - COMPLETE STUDY NOTES

Annotations • Dependencies • Lifecycle • Selenium Test Design

Learning goal

Understand not only what each TestNG annotation does, but when to use it, how TestNG decides whether a test passes or skips, and how to organize Selenium setup and cleanup without fragile timing or shared-state problems.

Prepared from the supplied "TestNG class 2" lecture notes, with beginner-friendly explanations, corrected terminology, practical examples, and industry-oriented guidance.

Code examples use Java, TestNG, and Selenium WebDriver. Update selectors and environment configuration for your own application.

Contents and learning path

- 1. TestNG mental model and annotation basics
- 2. @Test attributes: description, enabled, priority, timeout, invocation
- 3. expectedExceptions: correct use and common misuse
- 4. dependsOnMethods, alwaysRun, and execution flow
- 5. Lifecycle annotations and execution order
- 6. Reading BeforeMethod / AfterMethod and BeforeClass / AfterClass examples
- 7. Selenium example: login and create Contact, Deal, and Task
- 8. @BeforeTest / @AfterTest and testng.xml
- 9. Multiple test classes, base classes, and WebDriver lifecycle design
- 10. Common mistakes and best practices
- 11. Hands-on exercises and interview revision
- 12. Quick-reference sheet and official references

> **A note about the original source**
>
> The supplied class file contains hard-coded login values. They are deliberately not repeated in this guide. Use environment variables or a secret manager instead. The example application's URLs and locators are illustrative and may need updating.

1. TestNG: the mental model

TestNG is a Java testing framework. It discovers test methods through annotations, runs setup and cleanup methods at defined lifecycle points, handles dependencies and parameters, and produces test results. In a Selenium project, Selenium controls the browser; TestNG decides how the tests are organized and executed.

Concept	Meaning	Example
Test method	A Java method TestNG should execute as a test.	@Test public void loginWorks() { ... }
Test annotation	Metadata that tells TestNG how to run a method/class.	@Test(priority = 1, description = "Valid login")
Configuration method	Setup or cleanup that runs at a lifecycle point.	@BeforeMethod, @AfterClass
Suite	A top-level run, usually described by an XML file.	<suite name="RegressionSuite">
Test tag in XML	A named group of classes within a suite; not the same as one @Test method.	<test name="Chrome tests">
Test class	A Java class holding one or more test and configuration methods.	ContactTests.java

How a simple test works

```
import org.testng.annotations.Test;

public class LoginTests {
   @Test
   public void loginWithValidCredentials() {
      System.out.println("Login test is running");
   }
}
```

Line by line:

- import org.testng.annotations.Test; imports the annotation. Without the import, Java cannot resolve the short name @Test.
- @Test marks the method as a TestNG test method.
- The method should express a behavior you want to verify, such as a successful login, a validation message, or a saved record.
- System.out.println is only a demonstration. Real tests should perform actions and assert observable outcomes.

> **Important**
>
> TestNG does not require you to write a main() method for each test. The test runner (IDE, Maven Surefire, Gradle, or testng.xml) invokes the annotated tests.

2. @Test attributes

Attributes are named settings inside an annotation. For example, @Test(priority = 2, enabled = true) gives TestNG extra instructions about that test method. Attributes are not all interchangeable: priority affects ordering, dependencies express prerequisites, and timeout attributes impose time limits.

Attribute / annotation	What it means	Practical guidance
@Test	Marks a method (or class) as a test.	Use meaningful test names and assert expected behavior.
priority	Lower priority values are scheduled before higher values within relevant ordering rules.	Use sparingly. Prefer dependencies only when a true prerequisite exists.
enabled = false	Disables a test method.	Useful for temporarily disabling a test, but track why and when it will be re-enabled.
@Ignore	Can ignore a TestNG test/class.	For targeted disabling, @Test(enabled = false) is usually clearer.
description	Adds readable metadata to a test.	Use a short phrase describing the business behavior.
timeOut	Maximum time in milliseconds for a test invocation.	A timeout makes a test fail if it exceeds the configured limit; choose a realistic threshold.
invocationCount	Runs the same test method a specified number of times.	Useful for repetition checks; not a substitute for data-driven testing.
invocationTimeOut	Time limit for the repeated invocations as a whole.	Keep it compatible with invocationCount and the expected duration.
threadPoolSize	Allows invocations to run concurrently when used with invocationCount.	Use only when the test and its data are safe to run in parallel.
expectedExceptions	Declares an exception that the test is supposed to throw.	This does not ignore arbitrary exceptions; the expected exception is part of the assertion.
dependsOnMethods	Declares prerequisite test methods.	Use for a genuine execution dependency, not merely to force an arbitrary order.
alwaysRun	For @Test, allows a dependent test to run even if a dependency failed.	Be careful: downstream steps may not be valid if their prerequisite did not succeed.

2.1 description

```
@Test(description = "Verify login with valid credentials")
public void validLoginShouldOpenDashboard() {
    // Arrange test data, act, and assert the dashboard is visible.
}
```

A description improves reports and helps teammates understand the intent without opening the implementation. Keep it aligned with the test name and assertion.

2.2 enabled = false and @Ignore

```
@Test(enabled = false, description = "Temporarily disabled while feature is unavailable")
public void newCheckoutFlow() {
    // Test is not run by TestNG while disabled.
}
```

The source notes list both enabled = false and @Ignore as ways to skip a test. The distinction is scope and readability: enabled = false is explicit on an individual @Test method; @Ignore may be used at method or class level depending on placement. Disabled tests should not be forgotten. Track a defect/ticket and remove the disablement promptly.

> **Do not confuse a skipped test with a passed test**
>
> A disabled or dependency-skipped test has not demonstrated that the feature works. Reports should retain its skipped status.

2.3 priority

```
@Test(priority = 1)
public void login() { /* ... */ }

@Test(priority = 2)
public void openContacts() { /* ... */ }

@Test(priority = 3)
public void createContact() { /* ... */ }
```

The lecture notes use priority to control test order. Priorities default to 0; negative values are allowed. When methods have the same priority, TestNG applies its ordering rules, which are not a good foundation for designing dependent workflows. Avoid relying on alphabetical ordering as an application design rule.

> **Priority is not a dependency**
>
> A priority can influence which method is scheduled first, but it does not express that a test is valid only after another test succeeds. For that, use dependsOnMethods or make each test independent by preparing its own state.

2.4 timeOut

```
@Test(timeOut = 3000)
public void searchShouldFinishQuickly() {
    // Perform the search and assert the expected results.
}
```

timeOut is measured in milliseconds. With 3000, TestNG applies a three-second limit to the test invocation. The timeout does not mean “wait up to three seconds for a web element”; that is a Selenium wait concern. Use WebDriverWait for a condition and TestNG timeOut only as an outer guard against an unexpectedly long test.

2.5 invocationCount, invocationTimeOut, threadPoolSize

```
@Test(invocationCount = 4)
public void healthCheckCanBeRepeated() {
   // The method is invoked four times.
}

@Test(invocationCount = 4, invocationTimeOut = 15000)
public void repeatedCheckMustFinishWithinOverallLimit() {
   // Four invocations must fit within the configured total time limit.
}

@Test(invocationCount = 4, threadPoolSize = 2)
public void concurrentCheck() {
   // Invocations may be run by a pool of two threads.
}
```

How to interpret these settings:

- invocationCount = 4 means TestNG invokes the same method four times.
- invocationTimeOut = 15000 sets a 15-second time budget for the repeated invocations, rather than 15 seconds per individual call.
- threadPoolSize = 2 permits concurrent invocations, in conjunction with repeated execution.

> **Parallel-execution warning**
>
> Never share a single mutable WebDriver instance across parallel invocations. Browser state, current URL, cookies, and focused elements can interfere with one another. Use isolated driver instances (often one per test or per thread) and unique test data.

3. expectedExceptions: test the exception you expect

The source says expectedExceptions “ignores” an exception. This wording is misleading. TestNG marks the test as successful when the configured exception is thrown by the test method; if no expected exception is thrown, or a different exception occurs, the test fails.

```
import org.testng.annotations.Test;

public class ExceptionTests {
   @Test(expectedExceptions = NullPointerException.class)
   public void convertingNullToUpperCaseShouldThrow() {
      String name = null;
      name.toUpperCase(); // Expected NullPointerException occurs here.
   }
}
```

Explanation:

1. The method deliberately arranges a situation where calling toUpperCase() on a null reference is invalid.
2. TestNG observes the exception thrown from the test method.
3. Because the exception type matches NullPointerException, the test passes as an exception-behavior test.
4. The println statement after the failing line is never reached.

3.1 A realistic use case

Suppose a service rejects an invalid request by throwing IllegalArgumentException. An exception test can verify this contract. In normal UI automation, however, most negative user scenarios should assert the user-visible validation message, not expect Selenium to throw an exception.

```
@Test(expectedExceptions = IllegalArgumentException.class)
public void negativeQuantityIsRejected() {
    OrderCalculator.calculateTotal(-2, 100.0);
}
```

> **Do not “green” a broken Selenium locator**
>
> The source demonstrates expected NoSuchElementException after searching for a deliberately nonexistent element. That can prove Selenium throws when an element is missing, but it does not prove your application feature works. Do not use expectedExceptions to hide a wrong locator or unstable page.

3.2 Example: expected missing element versus expected application behavior

```
// Weak UI test: it can pass simply because the locator is wrong.
@Test(expectedExceptions = NoSuchElementException.class)
public void findElementWithUnknownId() {
    driver.findElement(By.id("not-a-real-element")).click();
}

// Better negative UI test: verify the product behavior users see.
@Test
public void invalidLoginShowsHelpfulError() {
    // Enter invalid credentials and submit the login form.
    String message = driver.findElement(By.cssSelector("[role='alert']")).getText();
    Assert.assertEquals(message, "Invalid username or password");
}
```

The second example is a pattern, not a universal selector. Use the actual accessible alert or validation element from your application DOM, and assert the exact contract agreed by the product team.

4. dependsOnMethods and alwaysRun

dependsOnMethods creates a declared prerequisite relationship between test methods. TestNG ensures prerequisite methods run before the dependent method. If a prerequisite fails, a dependent test is normally skipped because its precondition was not satisfied.

```
import org.testng.Assert;
import org.testng.annotations.Test;

public class ContactWorkflowTests {
    @Test
    public void login() {
        boolean loginSucceeded = true; // Replace with a real UI assertion.
        Assert.assertTrue(loginSucceeded, "Login should succeed");
    }

    @Test(dependsOnMethods = "login")
    public void createContact() {
        // Valid only if login passed and the authenticated session exists.
    }

    @Test(dependsOnMethods = "createContact")
    public void verifyContactDetails() {
        // Runs after successful createContact.
    }
}
```

This models a flow where createContact is not meaningful until login succeeds. If the login test fails, TestNG will normally mark createContact as skipped. The same applies transitively to verifyContactDetails.

4.1 Why an assertion matters

```
@Test
public void login() {
    Assert.assertEquals(12, 1000); // Deliberately fails.
    System.out.println("Login Test Cases"); // Not reached.
}
```

The lecture's Sample2 uses Assert.assertEquals(12, 1000). Because 12 is not equal to 1000, TestNG reports the login test as failed; it does not continue to the print statement. That failure is then relevant to dependent tests.

4.2 What alwaysRun = true really means

```
@Test(dependsOnMethods = "login", alwaysRun = true)
public void collectDiagnostics() {
    // Example: collect a diagnostic artifact that is meaningful even if login failed.
}
```

For a TestNG test method, alwaysRun = true allows the dependent method to run even if a dependency failed. This can be useful for diagnostics or other work that remains valid after the prerequisite fails. It does not magically restore the authenticated state, and it is not a safe default for a business action that requires login.

> **Use dependencies with care**
>
> If every test needs a logged-in browser, consider logging in during controlled setup or creating an independent authenticated state for each test. Tests that depend on other tests can become brittle: one failure causes a chain of skips.

4.3 Priority or dependency?

Situation	Prefer	Reason
Two independent tests just happen to be easier to read in an order.	Neither dependency nor fragile ordering; independent tests.	The suite should still work if execution order changes.
A test must use an account created by a prior test.	dependsOnMethods, or better: arrange/create the account in setup.	A real prerequisite exists.
Run setup before every test.	@BeforeMethod.	Lifecycle config is clearer than making tests depend on a setup test.
Run a cleanup/diagnostic even when an assertion fails.	@AfterMethod or @AfterClass cleanup; use configuration alwaysRun where appropriate.	Cleanup belongs in lifecycle configuration, not a normal business test.

5. TestNG lifecycle annotations

The lecture groups annotations into preconditions, the @Test annotation, and postconditions. In practice, a lifecycle annotation defines the scope around which setup or cleanup runs.

Annotation	Scope	Typical responsibility
@BeforeSuite	Once before the TestNG suite.	Start suite-wide resources or initialize global reporting.
@BeforeTest	Before test methods in classes associated with a <test> tag in testng.xml.	Prepare a resource for a configured group of classes, when scope is understood.
@BeforeClass	Before the first test method in the current class.	Create a class-level resource or navigate to a known starting state.
@BeforeMethod	Before each @Test method invocation.	Reset state, start a fresh browser, or prepare test-specific data.
@Test	The test itself.	Perform an action and assert expected behavior.
@AfterMethod	After each @Test method invocation.	Capture failure artifacts, clear state, or close a per-test browser.
@AfterClass	After all test methods in the current class.	Dispose class-level resources.
@AfterTest	After test methods associated with a <test> XML tag.	Dispose resources initialized at matching scope.
@AfterSuite	After suite execution.	Close suite-wide resources and finalize reporting.

5.1 Memorize the scope ladder

```
@BeforeSuite
  @BeforeTest
    @BeforeClass
      @BeforeMethod
        @Test method
      @AfterMethod
    @AfterClass
  @AfterTest
@AfterSuite
```

This is a useful mental model for one common, non-parallel suite. Exact invocation details can be affected by XML layout, inherited configuration methods, multiple classes, factories, and parallel settings. When setup is critical, confirm behavior by running a tiny example and inspecting the TestNG output.

5.2 @BeforeMethod and @AfterMethod

These are per-test hooks. If the class has three test methods and each one is invoked once, @BeforeMethod normally runs three times and @AfterMethod normally runs three times.

```
import org.testng.annotations.*;

public class Sample3 {
   @BeforeMethod public void beforeEachTest() { System.out.println("Before Method"); }
   @Test public void testCase1() { System.out.println("Test 1"); }
   @Test public void testCase2() { System.out.println("Test 2"); }
   @Test public void testCase3() { System.out.println("Test 3"); }
   @AfterMethod public void afterEachTest() { System.out.println("After Method"); }
}
```

The exact ordering among test methods should not be inferred from the method names. The reliable observation is that each selected test invocation is surrounded by the configuration methods.

Illustrative output pattern:

```
Before Method
Test 1
After Method
Before Method
Test 2
After Method
Before Method
Test 3
After Method
```

5.3 @BeforeClass and @AfterClass

These are class-level hooks. The setup method is called before the first test method in that class, and the teardown method after the class's test methods have run. If a class contains many test methods, class setup avoids repeated initialization—but it also means those tests may share browser/session state.

```
import org.testng.annotations.*;

public class Sample4 {
   @BeforeClass public void setUpClass() { System.out.println("Before class"); }
   @Test public void testCase1() { System.out.println("Test 1"); }
   @Test public void testCase2() { System.out.println("Test 2"); }
   @Test public void testCase3() { System.out.println("Test 3"); }
   @AfterClass public void tearDownClass() { System.out.println("After class"); }
}
```

> **Choose the lifecycle by desired isolation**
>
> Use @BeforeMethod when every test needs a clean starting point. Use @BeforeClass when sharing a resource is intentional and you understand how tests can affect one another. Creating a driver once per class is not automatically better than creating one per test.

5.4 @BeforeSuite and @AfterSuite

Although these annotations are listed in the lecture's lifecycle summary, the source does not give a code example for them. They are suite-level hooks—typically suitable for initializing suite-wide reporting, reading

shared configuration, or closing suite-wide resources. Do not use them for mutable WebDriver state shared across parallel tests.

6. Selenium workflow: login, create Contact, Deal, and Task

The supplied Selenium example opens a CRM login page once, then runs test methods that navigate to Contacts, Deals, and Tasks, create records, and save them. It closes the browser after the class. The overall idea is class-scoped setup and teardown; the implementation has issues that should be corrected before using it in a real automation suite.

6.1 What the original example demonstrates

- A WebDriver field lets multiple test methods in one class refer to the same browser instance.
- @BeforeClass runs the login/setup method before the class's test methods.
- Each @Test method navigates to a module, opens a creation form, enters data, and clicks Save.
- @AfterClass calls driver.quit() to close the browser session.

6.2 Problems to correct

Original pattern	Risk	Better approach
Hard-coded email/password in Java.	Credentials can leak into Git, build logs, and shared documents.	Read from environment variables or a secret manager.
Thread.sleep(7000) throughout.	Slow and flaky when the app is faster/slower than expected.	Use WebDriverWait for a specific condition.
No meaningful assertion after Save.	A click can occur without proving the record was saved.	Assert a success message or find the created record.
One browser state is shared by all tests.	Navigation or a failed test can affect the next test.	Use independent setup or deliberately controlled class-level state.
Implicit wait set to 30 seconds and fixed sleeps also used.	Unnecessary delay and confusing wait behavior.	Prefer explicit waits and keep the wait strategy consistent.
WebDriver may not be closed if setup fails before tests run.	Resource leakage can leave browser processes running.	Use defensive cleanup and framework-level resource management.

6.3 Safer illustrative pattern

This is a teaching template. The URL, locators, confirmation selector, and app workflow must be adapted to the application under test. This example intentionally does not include real credentials.

```
package tests;

import java.time.Duration;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;
import org.testng.Assert;
import org.testng.annotations.*;

public class CrmSmokeTests {
   private WebDriver driver;
   private WebDriverWait wait;

   @BeforeClass
   public void setUp() {
      String email = System.getenv("CRM_EMAIL");
      String password = System.getenv("CRM_PASSWORD");
      if (email == null || password == null)
         throw new IllegalStateException("Set CRM_EMAIL and CRM_PASSWORD first.");

      driver = new ChromeDriver();
      wait = new WebDriverWait(driver, Duration.ofSeconds(10));
      driver.manage().window().maximize();
      driver.get("https://your-test-app.example/login");
      driver.findElement(By.id("email")).sendKeys(email);
      driver.findElement(By.id("password")).sendKeys(password);
      driver.findElement(By.xpath("//button[normalize-space()='Login']")).click();
      wait.until(ExpectedConditions.visibilityOfElementLocated(
         By.cssSelector("[data-testid='dashboard']")));
   }

   @Test(description = "Create a contact and verify success")
   public void addNewContact() {
      wait.until(ExpectedConditions.elementToBeClickable(
         By.cssSelector("a[href='/contacts']"))).click();
      wait.until(ExpectedConditions.elementToBeClickable(
         By.xpath("//button[normalize-space()='Create']"))).click();
      driver.findElement(By.id("first-name")).sendKeys("Automation");
      driver.findElement(By.id("last-name")).sendKeys("Contact");
      driver.findElement(By.xpath("//button[normalize-space()='Save']")).click();

      WebElement confirmation = wait.until(ExpectedConditions.visibilityOfElementLocated(
         By.cssSelector("[role='status']")));
      Assert.assertTrue(confirmation.getText().contains("saved"),
         "A success confirmation should be displayed.");
   }

   @AfterClass(alwaysRun = true)
   public void tearDown() {
      if (driver != null) driver.quit();
   }
}
```

Walkthrough of the important code

1. WebDriver driver stores the browser controller. It is an instance field, not static, so different test class instances do not automatically share one global reference.

2. WebDriverWait waits for conditions such as visibility or clickability. It waits only as long as needed, up to the configured limit.
3. System.getenv reads credentials injected by the machine or CI pipeline. If they are missing, setup fails early with a clear message instead of attempting a login with null values.
4. @BeforeClass logs in once for this class. This is efficient for a compact smoke-test class but deliberately shares the authenticated session across its test methods.
5. @Test drives a user flow and then asserts a visible confirmation. The assertion, not the click alone, decides whether the expected behavior was observed.
6. @AfterClass(alwaysRun = true) attempts cleanup even when an earlier configuration/test result would otherwise interfere; the null check protects against setup failing before driver creation.

> **One more design decision**
>
> If the Contact, Deal, and Task tests must be independent, start from a clean state in @BeforeMethod or use separate test fixtures/accounts. Do not depend on a prior test having left the browser on a particular page.

6.4 Example for Deals and Tasks

Repeat the same Arrange -> Act -> Assert structure for each feature: open the Deals module, create a uniquely named deal, save, then assert the deal appears; open Tasks, create a uniquely named task, save, then assert the task appears. Generate unique names (for example, include a timestamp or UUID) to avoid duplicate-record collisions in repeated CI runs.

7. @BeforeTest / @AfterTest and testng.xml

The source explains that @BeforeTest and @AfterTest are associated with the classes listed inside a <test> tag in testng.xml. The important distinction is that XML <test> is a TestNG configuration grouping; it is not equivalent to a single Java method annotated with @Test.

7.1 Understanding the XML hierarchy

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="CrmSuite">
  <test name="SmokeTests">
    <classes>
      <class name="tests.ContactTests" />
      <class name="tests.DealTests" />
      <class name="tests.TaskTests" />
    </classes>
  </test>
</suite>
```

How to read it:

- <suite> names the overall run. One suite can contain multiple <test> tags.
- <test> names a configured collection of classes. Parameters and execution configuration may be scoped here.
- <classes> contains the Java classes TestNG should discover.
- <class name="tests.ContactTests" /> uses the fully qualified class name: package plus class name.

7.2 Pair @BeforeTest and @AfterTest carefully

```
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;

public class EnvironmentHooks {
  @BeforeTest
  public void prepareEnvironmentForXmlTest() {
    System.out.println("Runs before test methods associated with this XML <test>");
  }

  @AfterTest(alwaysRun = true)
  public void cleanEnvironmentAfterXmlTest() {
    System.out.println("Runs after test methods associated with this XML <test>");
  }
}
```

A common mistake is to assume an inherited setup method with a static WebDriver field will always create exactly one browser shared safely across every class. TestNG configuration invocation depends on class membership and configuration inheritance; the result should be verified in the actual suite. For browser isolation, an explicit BaseTest design with well-defined driver ownership is easier to reason about.

8. Multiple classes, base test classes, and clean design

The source's final example defines a BasePage class with @BeforeTest login code and @AfterTest driver.quit(), then makes ContactPage, DealsPage, and TaskPage extend it. It also lists BasePage itself in testng.xml. This demonstrates inheritance and shared setup, but it mixes responsibilities.

8.1 BasePage versus BaseTest

A Page Object represents a page or component and contains locators and user actions. A BaseTest (or test fixture) owns test lifecycle concerns such as starting/closing WebDriver. Naming setup code BasePage can blur these responsibilities. Prefer a structure such as:

```
src/test/java/
 base/
  BaseTest.java         // Driver lifecycle and common test setup
 pages/
  LoginPage.java        // Login locators and actions
  ContactsPage.java     // Contact page locators and actions
  DealsPage.java        // Deal page locators and actions
  TasksPage.java        // Task page locators and actions
 tests/
  ContactTests.java     // Assertions about contact workflows
  DealTests.java
  TaskTests.java
 testng.xml
```

8.2 Why static WebDriver is risky

- A static driver is shared at class level, which can create accidental sharing between test classes or threads.
- When running tests in parallel, one test can change the URL or close the browser while another test is using it.
- A shared browser also allows cookies, local storage, and form state to leak between tests.
- Prefer instance-owned drivers for serial tests, or a deliberate ThreadLocal/WebDriver factory design for parallel execution.

8.3 A simple BaseTest template

```
package base;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public abstract class BaseTest {
   protected WebDriver driver;

   @BeforeMethod
   public void startBrowser() {
      driver = new ChromeDriver();
      driver.manage().window().maximize();
      driver.manage().timeouts().implicitlyWait(Duration.ZERO);
   }

   @AfterMethod(alwaysRun = true)
   public void closeBrowser() {
      if (driver != null) {
         driver.quit();
         driver = null;
      }
   }
}
```

This template creates a fresh browser for every @Test invocation and closes it afterward. It deliberately sets implicit wait to zero because the tests should use explicit waits. In a real framework, add configuration for browser type, environment URL, screenshots on failure, and driver options. If the test requires login, log in during that test's setup or through an API/fixture that creates isolated state.

8.4 Test class using the base

```
package tests;

import base.BaseTest;
import org.openqa.selenium.By;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.Test;
import java.time.Duration;

public class ContactTests extends BaseTest {
  @Test
  public void contactFormShouldBeAvailable() {
    driver.get("https://your-test-app.example/login");
    // Authenticate using safe test credentials or a test fixture.
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    boolean createButtonVisible = wait.until(
      ExpectedConditions.visibilityOfElementLocated(
        By.xpath("//button[normalize-space()='Create']")))
      .isDisplayed();
    Assert.assertTrue(createButtonVisible,
      "Create button should be visible after navigating to Contacts.");
  }
}
```

The example emphasizes where a test extends BaseTest and how assertions fit in. The application-specific login/navigation steps must be implemented for your environment; do not copy the placeholder URL and assume it is a live service.

8.5 XML should list test classes, not the base class

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="CrmRegressionSuite">
  <test name="Smoke">
    <classes>
      <class name="tests.ContactTests" />
      <class name="tests.DealTests" />
      <class name="tests.TaskTests" />
    </classes>
  </test>
</suite>
```

BaseTest is a superclass that provides lifecycle behavior; it normally should not be listed as a business test class when it contains no @Test cases of its own. The test classes inherit its setup/cleanup methods.

9. Best practices checklist

- Write assertions for business outcomes. A click or a println is not proof that a feature worked.
- Use expectedExceptions only when throwing that specific exception is the behavior under test.
- Use dependsOnMethods only for real prerequisites; keep tests independent where possible.
- Do not use alwaysRun on a downstream business action if its prerequisite failed and the action is no longer valid.
- Use @BeforeMethod / @AfterMethod for per-test isolation; use class-level lifecycle only intentionally.
- Avoid Thread.sleep. Wait for visibility, clickability, URL changes, or a domain-specific condition with WebDriverWait.

- Avoid hard-coded passwords, access tokens, and private environment values. Inject them through secure configuration.
- Avoid static WebDriver for normal test execution, especially when parallel execution is possible.
- Use unique test data and clean it up where practical. Make cleanup resilient with alwaysRun configuration where appropriate.
- Give tests descriptive names and descriptions. A failing report should tell a reader which behavior broke.
- Use realistic timeouts. An unrealistically small timeout creates flakiness; an overly large timeout slows feedback.
- When a configuration method fails, debug the setup failure before treating every skipped test as an individual product defect.
- Do not assume test-method ordering from alphabetic names. Use explicit dependencies only when needed.
- Run the suite from testng.xml / Maven in CI as well as from the IDE so local and pipeline execution stay consistent.

Common mistakes: symptom -> likely cause -> fix

Symptom	Likely cause	Fix
A test passes when an element is absent.	expectedExceptions expects Selenium failure instead of expected product behavior.	Assert the business-level error or empty state.
Many tests are skipped after one failure.	They depend on the failed test method.	Remove unnecessary test dependencies; create state in setup or fixtures.
A test unexpectedly runs after login fails.	alwaysRun = true bypasses a failed dependency.	Remove it for business steps that require successful login.
Tests fail intermittently after Thread.sleep.	Application readiness varies by machine/load.	Wait for a specific condition.
Parallel runs close each other's browser.	Static/shared WebDriver or shared test data.	Give each worker an isolated driver and unique data.
A record click appears successful but no record exists.	No assertion on saved state/confirmation.	Assert success and, when necessary, query the record afterward.
Browser stays open after a failed test.	Cleanup did not run or does not handle partial setup.	Use @AfterMethod(alwaysRun = true), null checks, and driver.quit().

10. Hands-on exercises

Exercise 1: descriptions and disabled tests

Create a class with three @Test methods. Add a meaningful description to each method. Disable one method with enabled = false. Run the class and identify which methods are passed and which are skipped.

Exercise 2: timeout versus explicit wait

Write a test that waits up to 10 seconds for a page heading. Add a reasonable TestNG timeOut to guard the whole test. Explain why these two time limits serve different purposes.

Exercise 3: exception contract

Write a unit-level test for a method that throws IllegalArgumentException for invalid input. Use expectedExceptions. Then write a UI test that checks a validation message for an invalid form input without expecting a Selenium locator exception.

Exercise 4: dependencies

Create login, createContact, and verifyContact tests. Make createContact depend on login, and verifyContact depend on createContact. Intentionally fail login and observe the skip cascade. Then discuss how to redesign the tests so each one creates its own prerequisite state.

Exercise 5: lifecycle tracing

Add @BeforeSuite, @BeforeTest, @BeforeClass, @BeforeMethod, @Test, @AfterMethod, @AfterClass, @AfterTest, and @AfterSuite methods with print statements. Run two classes inside one XML <test> tag, then experiment with two <test> tags. Record the actual behavior of your TestNG version and execution configuration.

Exercise 6: create three records

Create ContactTests, DealTests, and TaskTests. Use a shared BaseTest for browser setup and teardown, unique test data, explicit waits, and assertions after Save. Run them using testng.xml. Do not list BaseTest as a business test class unless it intentionally contains its own tests.

11. Interview revision questions

Q. What is the difference between priority and dependsOnMethods?

A. Priority influences execution ordering. dependsOnMethods establishes a prerequisite and normally skips the dependent test when the prerequisite fails.

Q. Does expectedExceptions ignore exceptions?

A. No. It makes the expected exception part of the test contract. The test fails if the expected exception is not thrown or the wrong exception is thrown.

Q. What does timeOut = 5000 mean?

A. A maximum test-invocation duration of 5,000 milliseconds, i.e. five seconds.

Q. What is invocationCount?

A. It asks TestNG to invoke the same test method a specified number of times.

Q. What is the risk of threadPoolSize?

A. Invocations can execute concurrently, so shared WebDriver, mutable fields, test data, and application state must be isolated.

Q. How often does @BeforeMethod run?

A. Before each selected test method invocation, including repeated invocations under applicable execution settings.

Q. When would you use @BeforeClass?

A. When a setup resource is intentionally shared across test methods in the class.

Q. What does @BeforeTest refer to?

A. It is associated with test methods belonging to classes configured under a testng.xml <test> tag, not each individual @Test method.

Q. Why is Thread.sleep discouraged?

A. It always waits for the full duration and may still be too short under load. Explicit condition-based waits are usually faster and more reliable.

Q. Why should WebDriver not usually be static?

A. Static shared browser state causes cross-test interference and is unsafe for parallel execution unless intentionally isolated/managed.

Q. What does alwaysRun = true do on a dependent @Test?

A. It permits the dependent method to run even if a dependency failed; use it only if that dependent task remains meaningful.

Q. What should be in BaseTest vs a Page Object?

A. BaseTest owns test lifecycle and common fixture setup; Page Objects represent page structure and user actions. Keep the responsibilities separated.

12. Quick-reference sheet

Need	Use	Remember
Mark a method as a test	@Test	Write a meaningful assertion.
Describe the test	@Test(description = "...")	Descriptions appear in test metadata/reports.
Temporarily disable a test	@Test(enabled = false)	Track the reason and restore it promptly.
Set a time budget	@Test(timeOut = 5000)	Milliseconds; outer test timeout.
Repeat a test	@Test(invocationCount = 4)	Repeated test data must be safe.
Expected exception behavior	@Test(expectedExceptions = SomeException.class)	Only use for intentionally expected exceptions.
Depend on a prerequisite test	@Test(dependsOnMethods = "login")	Failed dependency normally means a skipped dependent test.
Setup/cleanup per test	@BeforeMethod / @AfterMethod	Best starting point for independent UI tests.
Setup/cleanup per class	@BeforeClass / @AfterClass	Shared browser/session state is a trade-off.
Configure classes for execution	testng.xml	List test classes by fully qualified name.

Corrections and clarifications made to the lecture wording

- expectedExceptions is not an “ignore exception” switch; it asserts an expected exception type.
- alwaysRun = true on a dependent @Test can cause that test to run even after a prerequisite failed; this is not generally safe for dependent user flows.
- priority is not a substitute for proper test setup or a declared prerequisite.
- A missing Selenium element is normally a test failure to investigate, not an exception to suppress. Test the intended application response instead.
- The lecture’s fixed sleeps and embedded credentials have been replaced with explicit-wait examples and environment-variable configuration.
- The base-class example has been reframed as BaseTest rather than BasePage to separate test fixture/lifecycle responsibilities from Page Object responsibilities.
- Scope descriptions for @BeforeTest and other lifecycle hooks follow TestNG’s official documentation. With inheritance or parallel execution, verify configuration invocation in the actual suite.

References

Primary source: user-provided “TestNG class 2.txt”. Official docs: TestNG guide https://testng.org/documentation.html | @Test API https://javadoc.io/doc/org.testng/testng/latest/org/testng/annotations/Test.html | Selenium waits https://www.selenium.dev/documentation/webdriver/waits/. Use documentation matching your project version.